Roll No: _____ **Date:** _____

Experiment-10
Deadlock

Theory

Deadlock: A deadlock is a situation in an operating system where two or more processes are permanently blocked, each waiting for a resource that is held by another process.

Conditions that lead to Deadlock:

1. **Mutual Exclusion** - Resources cannot be shared; only one process can use a resource at a time.
2. **Hold and Wait** - A process holding at least one resource is waiting to acquire additional resources held by other processes.
3. **No Preemption** - Resources cannot be forcibly taken away from a process; they must be released voluntarily.
4. **Circular Wait** - A circular chain of processes exists, where each process is waiting for a resource held by the next process in the chain.

Methods to Handle Deadlock:

1. **Deadlock Prevention** - Ensure that at least one of the four necessary conditions never occurs.
2. **Deadlock Avoidance** - Dynamically examine resource allocation to ensure a circular wait never occurs (e.g., Banker's Algorithm).
3. **Deadlock Detection and Recovery** - Allow deadlocks to occur, detect them using an algorithm, and then recover (e.g., terminate or preempt processes).

Roll No: _____ **Date:** _____

Program 1: Banker's Algorithm for Deadlock Detection

Aim: Implementation of Banker's Algorithm for Deadlock Detection.

Description

This algorithm checks if the system is in a deadlock state. It uses the allocation, request, and available matrices to simulate resource allocation.

- If all processes can finish with available resources, the system is safe.
- If some remain unfinished, those processes are deadlocked.

Algorithm

1. Input number of processes (n) and resources (m).
2. Read Allocation, Request, and Available matrices.
3. Initialize:
   - `work = avail`
   - `finish[i] = 0`
4. Repeat for each process:
   - If `finish[i] == 0` and for all `j`, `(req[i][j] - alloc[i][j]) <= work[j]`, then:
     - Mark `finish[i] = 1`.
     - Add `alloc[i][j]` to `work[j]`.
     - Add process to safe sequence.
5. After checking all processes:
   - If any `finish[i] == 0`, system is in deadlock.
   - Else system is in safe state and print the safe sequence.

Source Code

```c
#include <stdio.h>

int main() {
    int n, m, i, j, k;

    // Input number of processes and resources
    printf("Enter number of processes and resources: ");
    scanf("%d %d", &n, &m);

    int alloc[n][m], req[n][m], avail[m], finish[n], seq[n], idx=0;
```

```
11
12      // Input Allocation Matrix
13      printf("Enter Allocation Matrix (%d x %d):\n", n, m);
14      for(i=0; i<n; i++) {
15          for(j=0; j<m; j++) {
16              scanf("%d", &alloc[i][j]);
17          }
18      }
19
20      // Input Request Matrix
21      printf("Enter Request Matrix (%d x %d):\n", n, m);
22      for(i=0; i<n; i++) {
23          for(j=0; j<m; j++) {
24              scanf("%d", &req[i][j]);
25          }
26      }
27
28      // Input Available Resources
29      printf("Enter Available Resources (%d values): ", m);
30      for(i=0; i<m; i++) {
31          scanf("%d", &avail[i]);
32      }
33
34      // Initialize work = available and finish[i] = 0
35      int work[m];
36      for(i=0; i<m; i++) {
37          work[i] = avail[i];
38      }
39      for(i=0; i<n; i++) {
40          finish[i] = 0;
41      }
42
43      // Deadlock detection loop
44      for(k=0; k<n; k++) {
45          for(i=0; i<n; i++) {
46              if(finish[i] == 0) { // If process not finished
47                  for(j=0; j<m; j++) {
48                      if(req[i][j] - alloc[i][j] > work[j]) {
49                          break; // Cannot satisfy this process
50                      }
51                  }
52                  if(j == m) { // If all resource needs can be satisfied
53                      for(j=0; j<m; j++) {
54                          work[j] += alloc[i][j]; // Release allocated
    resources
55                      }
56                      finish[i] = 1; // Mark process as finished
57                      seq[idx++] = i; // Add to safe sequence
58                  }
59              }
60          }
61      }
62
63      // Check for deadlock
64      int dead = 0;
65      for(i=0; i<n; i++) {
66          if(finish[i] == 0) {
```

```
            dead = 1;
        }
    }

    // Print result
    if(dead) {
        printf("\nSystem is in Deadlock.\nDeadlocked Processes: ");
        for(i=0; i<n; i++) {
            if(finish[i] == 0) {
                printf("P%d ", i+1);
            }
        }
        printf("\n");
    } else {
        printf("\nSystem is in Safe State.\nSafe Sequence: ");
        for(i=0; i<n; i++) {
            printf("P%d ", seq[i] + 1);
        }
        printf("\n");
    }
    return 0;
}
```

Output

```
student@cselab:~$ gcc banker.c
student@cselab:~$ ./a.out
Enter number of processes and resources: 5 3
Enter Allocation Matrix (5 x 3):
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Request Matrix (5 x 3):
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2
Enter Available Resources (3 values): 0 0 0

System is in Safe State.
Safe Sequence: P1 P3 P4 P5 P2
```

(Note: Matrix inputs in the terminal example are assumed based on standard Banker's algorithm test sets as OCR data for exact inputs was truncated.)

Program 2: Banker's Algorithm for Deadlock Avoidance

Aim: Implementation of Banker's Algorithm for Deadlock Avoidance.

Description

The Banker's Algorithm is a deadlock avoidance algorithm. It ensures that the system stays in a safe state before allocating resources.

- The algorithm checks if all processes can finish with available resources using the Need, Allocation, and Available matrices.
- If all processes can finish in some order, system is safe.
- If some remain unfinished, system is unsafe (possible deadlock).

Algorithm

1. **Input:**
   - Number of processes (n) and resources (m).
   - Allocation, Maximum, and Available matrices.
2. **Compute Need Matrix:**
   - `Need[i][j] = Max[i][j] - Allocation[i][j]`
3. **Initialize:**
   - `Finish[i] = 0` for all processes.
   - `Work[] = Available[]`
4. **Find a Safe Sequence:**
   - Repeat for all processes:
   - If `Finish[i] == 0` and `Need[i][j] <= Work[j]` for all `j`, then:
     - Allocate resources: `Work[j] = Work[j] + Allocation[i][j]`
     - Mark `Finish[i] = 1`
     - Add process Pi to safe sequence
5. **Check Safety:**
   - If all `Finish[i] == 1`, Safe State (print safe sequence).
   - Else Unsafe State (deadlock may occur).

Source Code

```c
#include <stdio.h>

int main() {
    int n, m, i, j, k;

    // Input number of processes and resources
    printf("Enter number of processes and resources: ");
    scanf("%d %d", &n, &m);

    int alloc[n][m], max[n][m], need[n][m], avail[m], finish[n], seq[n], idx = 0;

    // Input Allocation Matrix
    printf("Enter Allocation Matrix (%d x %d):\n", n, m);
    for(i=0; i<n; i++) {
        for(j=0; j<m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    // Input Maximum Matrix
    printf("Enter Maximum Matrix (%d x %d):\n", n, m);
    for(i=0; i<n; i++) {
        for(j=0; j<m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    // Input Available Resources
    printf("Enter Available Resources (%d values): ", m);
    for(i=0; i<m; i++) {
        scanf("%d", &avail[i]);
    }

    // Calculate Need Matrix: Need = Max - Allocation
    for(i=0; i<n; i++) {
        for(j=0; j<m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    // Print Need Matrix
    printf("\nNeed Matrix:\n");
    for(i=0; i<n; i++) {
        for(j=0; j<m; j++) {
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }

    // Initialize Finish array and Work vector
    for(i=0; i<n; i++) {
        finish[i] = 0;
    }

```

```c
    int work[m];
    for(i=0; i<m; i++) {
        work[i] = avail[i];
    }

    // Banker's Algorithm: Find safe sequence
    for(k=0; k<n; k++) {
        for(i=0; i<n; i++) {
            if(finish[i] == 0) { // If process not finished
                for(j=0; j<m; j++) {
                    if(need[i][j] > work[j]) {
                        break; // Cannot satisfy this process
                    }
                }
                if(j == m) { // If all resources can be allocated
                    for(j=0; j<m; j++) {
                        work[j] += alloc[i][j]; // Release allocated resources
                    }
                    finish[i] = 1; // Mark process as finished
                    seq[idx++] = i; // Add to safe sequence
                }
            }
        }
    }

    // Check if system is safe or unsafe
    int unsafe = 0;
    for(i=0; i<n; i++) {
        if(finish[i] == 0) {
            unsafe = 1;
        }
    }

    if(unsafe) {
        printf("\nSystem is in UNSAFE State (Deadlock may occur).\nUnfinished Processes: ");
        for(i=0; i<n; i++) {
            if(finish[i] == 0) {
                printf("P%d ", i+1);
            }
        }
        printf("\n");
    } else {
        printf("\nSystem is in SAFE State.\nSafe Sequence: ");
        for(i=0; i<n; i++) {
            printf("P%d ", seq[i] + 1);
        }
        printf("\n");
    }

    return 0;
}
```

Output

```
student@cselab:~$ gcc bankera.c
student@cselab:~$ ./a.out
Enter number of processes and resources: 5 3
Enter Allocation Matrix (5 x 3):
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Matrix (5 x 3):
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources (3 values): 3 3 2

Need Matrix:
7 4 3
0 2 0
6 0 0
0 1 1
4 3 1

System is in SAFE State.
Safe Sequence: P2 P4 P5 P1 P3
```